Exporter CRM — Build Prompts (one per chunk)

Sep 30, 2026 · @Tech Services

One prompt per chunk from the UI Build Chunks doc, self-contained enough to hand to a builder directly. Every file path and entity name below was confirmed by reading the actual codebase in this session, not assumed. Where something genuinely isn't decided yet (a vendor, an exact field, a naming choice), the prompt says so explicitly instead of inventing an answer — build the surrounding structure and leave that one piece open, don't guess it.

Tech stack (verified by reading the actual config files, not assumed)

Backend: Python 3.12, FastAPI 0.141, SQLAlchemy 2.0 (async), Alembic 1.14 for migrations, asyncpg, Pydantic 2.10 + pydantic-settings, structlog for logging, httpx for outbound calls, Redis. Temporal (`temporalio` 1.31) and Kafka (`aiokafka`) are both present as dependencies but **off by default** — `TEMPORAL_ENABLED`, `KAFKA_ENABLED`, and `EVENT_CONSUMERS_ENABLED` all default to `false`, so the app runs against Postgres alone unless a chunk explicitly needs one of them (the live-dashboard event-bus work referenced in the FRD is the one place this matters). Metrics via Prometheus + `prometheus-fastapi-instrumentator`. Linting/typing: ruff (line-length 100, py312 target) and mypy (non-strict, `strict = false`). Module boundaries are enforced by `importlinter.ini`, not just convention — don't create a new cross-module import without checking it.

Frontend: React 18 + TypeScript, Vite 7, react-router-dom 7, TanStack Query 5 for data fetching, Tailwind CSS 3 (the exact token system every chunk above builds on), Radix UI primitives (dialog/dropdown/select/tabs/tooltip) for accessible components, react-hook-form + zod + `@hookform/resolvers` for forms, sonner for toasts, date-fns, lucide-react for icons. Package manager: pnpm 10. Testing: Vitest 4 + Testing Library, one `.test.tsx` alongside each page already — follow that pattern for every new page in these prompts, don't leave new pages untested.

API types are generated, not hand-maintained. `pnpm run generate:api` pulls the backend's live OpenAPI schema and runs `openapi-typescript` to produce `src/lib/api/schema.ts`. Run this after every backend route change in these prompts —

skipping it is exactly how this codebase's frontend types drifted from the backend once already.

Fonts: Inter is currently loaded via the `@fontsource-variable/inter` npm package, not a Google Fonts CDN link. Before wiring in Azeret Mono for Chunk 0, check whether an equivalent `@fontsource` package exists for it — that would match the existing loading pattern better than a new CDN import.

Infra (`docker-compose.yml`): Postgres 16 is the only service actually required for local dev. Also defined, all currently unused by default: Redis, Temporal + its UI, **two alternative event-bus stacks side by side** (Redpanda, and a separate Kafka + Zookeeper), a full observability stack (OpenTelemetry Collector, Jaeger, Prometheus, Grafana), pgAdmin, ngrok.

Chunk 0 — Design tokens

No Centrex reference — this is deliberately ANER's own brand kit, not Centrex's visual style (see the one-pager).

```
Extend frontend/tailwind.config.ts. Current state: brand.500 is #14b8a6,
ink.DEFAULT is #0f172a, fontFamily.sans is ['Inter', ...]. Change to ANER's
brand kit: typeface Azeret Mono (add as a Google Fonts import or self-hosted
font source, then add to fontFamily alongside or in place of the existing
sans entry), and palette cream #DFCDBD, sage #C4DCC9, off-white #F1F1F1,
near-black #181818.

Map the four brand colors onto the EXISTING semantic token groups already
defined in that file — ink, surface, stage, status, risk — by changing their
hex values. Do not invent new token names or rebuild the token structure;
it's already well-designed.

Do NOT decide whether Azeret Mono replaces Inter everywhere or is scoped to
specific elements (headings/data, body stays humanist sans) — that's an open
decision for whoever owns the brand kit. Build the font-loading mechanism so
either answer is a one-line change.

After the token change, do a visual pass over every existing page (HomePage,
the company/exporter list and detail pages, DealDetailPage, PipelinePage,
QualificationCriteriaPage, LoginPage) and confirm nothing becomes hard to
read — a palette swap changes contrast ratios everywhere at once.

Build this first. Every other chunk should be built against the final tokens,
not restyled afterward.
```

Chunk 0.5 — Home landing screen

No direct Centrex reference — this page is ANER's own "what needs my attention" pattern, not modeled on a specific Centrex screen.

```
Extend frontend/src/pages/HomePage.tsx. Do not create a new file or change
the router — "/" already renders this page after login via ProtectedRoute in
frontend/src/routes/AppRouter.tsx, and LoginPage.tsx is unchanged.

Current page: a personalized greeting, then FollowUpsDueCard,
CheckBacksDueCard, and PipelineSummaryCard (all imported from
@/modules/onboarding), gated by isStaffRole/useCurrentUser from
@/platform/auth. Keep all of that as-is.

Add a KPI strip above the existing cards, populated from the aggregation
endpoint built in the Chunk 1 prompt: new leads this period, new customers,
checks-cleared vs. opened, open value, average turnaround. Only wire this in
once that endpoint actually exists — do not fabricate numbers before it does.
Until then, use a brand-styled illustration or welcome treatment (Chunk 0's
tokens) as the placeholder. This page's own code comment already states the
principle: "there is no statistics endpoint, and this page does not pretend
there is one" — keep honoring that exactly.

Extend the role-aware behavior already present (the Add Company button only
shows for isStaffRole) rather than redesigning it: Compliance should
eventually see a pending-reviews count once a compliance queue exists;
Financer should eventually see a pending-decisions count once the Chunk 6
decision panel exists. Add these as new cards in the same grid, not by
changing the existing three.
```

Chunk 1 — Global Advances list

Centrex reference: p.5 §4 "Dynamic List Builder / Work Queues," p.6 §5 "Contact List / Operational Workbench."

```
New backend aggregation endpoint plus a new frontend page. Naming: use
"Advance" as the user-facing label; the underlying entity stays
Deal/Transaction in code — this is a display decision, not a rename, unless
the team explicitly decides otherwise.

Backend: no stats/summary endpoint exists anywhere in
backend/app/modules/onboarding today (confirmed by search). Build one. It
must compute server-side, not client-side: count of Deals per DealStage
(backend/app/modules/onboarding/domain/entities/deal_enums.py), new leads
created in a period, new Lead→Customer qualification conversions in a period
(qualification data lives in domain/entities/qualification.py and
qualification_enums.py), count of deals whose checks are fully clear vs.
total opened, open value, average turnaround.

Do NOT guess which field to sum for "open value." The current Deal entity
(domain/entities/deal.py) has no value or currency field at all today —
confirm whether that number comes from the invoices being built in the Chunk
2 prompt before building this part of the aggregation; build the rest of the
endpoint first if that's still undecided.

Frontend: new page, added to frontend/src/routes/AppRouter.tsx as its own
route (e.g. /advances) — NOT the "/" route, which stays HomePage per the
Chunk 0.5 prompt. Reuse the existing row-click-to-detail pattern already used
in DealDetailPage.tsx. Shows: the new KPI bar, one tab per DealStage value
with a live count, a filterable/sortable table, column customization, CSV
export.

Role gating: Operations, Compliance, Admin get full read and can open any
row. Financer gets read-only. Developer gets read-only with identifiers
masked — reuse the existing masking utility at frontend/src/platform/mask,
don't build a second one.
```

Chunk 1.5 — Company/Contact Dashboard

Centrex reference: p.9 §8 "Client Dashboard / Complete Audit History," p.14 §13 "Multiple Deals under One Client."

```
Before writing anything, read
frontend/src/modules/onboarding/pages/ExporterDetailPage.tsx in full — it may
already be this page, partially. Do not create a duplicate company-detail
page; extend what's there.

This page must be the single source of truth for, and host directly:
- The Lead→Prospect qualification gauge. Already built — reuse
QualificationCriteriaPage.tsx and the backend qualification.py /
qualification_enums.py, don't duplicate the logic.
- The company-level KYC/KYB/screening checklist. Already exists per
domain/entities/screening_review.py — surface it here if it isn't already.
- A "Request info" action that generates the tokenized link built in the
Chunk 4 prompt and lands submitted results back on this page.
- This company's own Advances, scoped to just them — reuse the Chunk 1 table
component, filtered by company_id, not a separate table implementation.
- The append-only Activity/History tab — reuse the existing append-only
pattern in domain/entities/exporter_lifecycle_history.py.

Why this page matters structurally: verification lives here, not on any
individual Advance, so redoing business with the same exporter later means
opening this page — the KYC/KYB already on file is right there, nothing gets
re-uploaded unless Compliance deliberately chooses to redo it.

Role gating: Operations/RM full read, can send info requests, can open this
company's Advances from here. Compliance runs/records verification decisions.
Financer read-only. Admin everything. Developer read-only masked.
```

Chunk 2 — Advance Detail (PO + Invoices)

Centrex reference: p.15 §14 "Deal Detail / Financing Terms," p.12 §11 "Documents and E-sign," p.14 §13 "Multiple Deals under One Client."

```
No PO or Invoice entity exists anywhere in the codebase today — confirmed by
searching backend/app/modules for class Invoice, class PurchaseOrder, class
Transaction: none found. This is new.

Backend: add a PO entity (or extend Deal in domain/entities/deal.py with PO
fields — not yet decided which; today Deal only has company_id, reference,
stage, withdrawal_reason, handed_over_at, and one DealBuyer) plus an Invoice
entity, many-to-one against the PO, since one PO can carry several invoices
(confirmed requirement: exporters sometimes split one order into multiple
invoices for tax reasons).

Document capture: the PO is created FROM an uploaded document, not a blank
form. Reuse the exact pattern already in domain/entities/crm_document.py —
relative storage key only (never a full provider path), the
DocumentScanStatus gate (PENDING_SCAN → AVAILABLE/QUARANTINED/SCAN_FAILED),
original filename kept as a column only, never in the storage key.

Do NOT pick a field-extraction vendor. Which tool reads the PO document into
structured fields is undecided (an "Invoice Analyser" was mentioned but its
actual capability is unconfirmed; AWS Textract was floated as an
alternative). Build the upload-then-confirm flow so extraction is a
pluggable, optional step: fields can arrive pre-filled by a future extractor,
or blank for a human to fill — either path must work today.

Frontend: new page, linked from the Company Dashboard's scoped Advances list
(Chunk 1.5 prompt), not created from a blank global "new Advance" button.
Shows: linked Buyer, stage, the Invoices list (add/remove), a Documents tab,
and a read-only summary that links back to the company's verification status
on the Company Dashboard — do not duplicate the checklist on this page.

Role gating: Operations/RM uploads the PO document, confirms/corrects
extracted fields, adds invoices and documents, advances stage (guarded —
check domain/entities/deal_enums.py for how permitted stage transitions are
currently enforced and follow that same pattern, don't invent a new one).
Compliance records verification decisions elsewhere (Chunk 1.5), cannot edit
PO or invoice fields here. Financer reads everything and records the decision
(Chunk 6 prompt). Developer read-only masked.
```

Chunk 3 — List Builder

Centrex reference: p.5 §4 "Dynamic List Builder / Work Queues."

```
Frontend: a form — list name, object type (Advances or Companies), a filter
builder, a field/column picker, a share-with picker. Its output must render
using the EXACT same table component built for the Chunk 1 Advances list —
don't build a second table implementation. Saved lists appear as additional
tabs on the Advances screen, alongside the built-in DealStage tabs, not as a
separate nav item.

Backend: a new "saved list" entity storing the filter/field/share definition
per user, plus a share target (self / team / everyone). Before building team-
sharing, check whether backend/app/platform/authorization already has a team
or group concept to reuse — not confirmed in this session; if none exists,
that's a prerequisite decision, don't invent an ad hoc one.

Every saved list must still apply the viewer's own role-based data masking at
render time, regardless of who built or shared it. Confirm the masking
mechanism backing today's exporter detail fields (frontend/src/platform/mask
handles some of it) is enforced server-side, not just in the frontend
component — a frontend-only permission check was the exact class of bug
already found and fixed once elsewhere in this codebase (a screening-review
endpoint), so verify this isn't repeating that pattern before shipping.
```

Chunk 4 — External intake forms + request queues

Centrex reference: p.3 §2 "External Application / Website Form," p.4 §3 "Data Sources, Field Mapping and Webhooks," p.12 §11 "Documents and E-sign."

```
Two separate public, unauthenticated pages reached via a tokenized link:
Exporter Intake (fuller data-entry form) and Buyer Acknowledgement (lighter —
view PO/invoice terms, click Acknowledge, not a data-entry form). Both are
requested from the Company Dashboard (Chunk 1.5 prompt), not from an
individual Advance.

Link distribution in the UI: three options — click-to-open, copy-to-
clipboard, and a QR code rendering the same link (client-side QR generation
is sufficient; no new backend endpoint needed just for the QR).

Backend: a token-based access mechanism for these two public routes is new —
searched backend/app/integrations/webhooks/inbound and found only an empty
__init__.py, no existing tokenized-link mechanism to reuse. Build one: a
signed, single-use or expiring token tied to the company (and, for the buyer
form, the specific Advance/PO).

Submitted files must auto-file into the document store using the exact same
crm_document.py pattern as Chunk 2, tagged with a DocumentSource value
identifying it arrived via the exporter or buyer form. Check
domain/entities/document_enums.py for the existing DocumentSource enum first
— extend it if a matching value isn't already there, don't assume one exists.

Internal side: two queue pages, Exporter Requests and Buyer Requests, each
listing every sent link with status (sent/opened/submitted/expired) and a
call/email action per row — reuse the Chunk 5 click-to-call/email pattern
here rather than building a separate action.

Role gating: Operations/RM generates links and works both queues. Compliance
views submitted results. Financer has no access to this page. Admin
everything. Developer read-only masked. The external party has no internal
role at all — access is the token, nothing else.
```

Chunk 5 — Click-to-call / click-to-email

Centrex reference: p.7 §6 "Call Tracking and Follow-up," p.8 §7 "Email from the CRM," p.11 §10 "Two-way Messaging" ("the channel is optional; the pattern is more important than SMS itself" — same reasoning applied to Teams here).

```
Not a new page. Add phone/email icon buttons to existing list rows (Chunk 1's Advances table, Chunk 4's request queues) and existing detail-page headers (DealDetailPage.tsx, the Company Dashboard from Chunk 1.5).

Clicking opens Microsoft Teams. Do NOT build real Teams Phone calling in this chunk — that requires Teams Phone (a Calling Plan or Direct Routing), a separate enterprise telephony setup from just deep-linking into Teams chat, and which of the two is actually wanted is still an open decision, not yet answered. Build the chat deep-link only (the teams.microsoft.com/l/chat pattern, addressed by the contact's email) and leave calling-plan integration as a follow-up once that decision is made.

After the Teams interaction, prompt the user to log an outcome note. Before writing it, confirm which history table the record type uses — exporter_lifecycle_history.py and the Deal's own dimension="deal" history rows are NOT the same table; write to whichever one already backs that specific record type, don't assume they're interchangeable.
```

Chunk 6 — Financer role + decision panel

Centrex reference: p.16 §15 "Underwriting Conditions, Decision Logic and Financier Submission."

```
Backend: add FINANCER to the UserRole enum in
backend/app/platform/authentication/models.py (currently ADMIN, COMPLIANCE,
OPERATIONS, API_USER, DEVELOPER — no FINANCER today). Follow the exact same
migration pattern already used to add DEVELOPER (referenced in that file's
own comments as auth_0002_developer_role) — don't invent a different
migration shape.

Gate the new panel with require_role(UserRole.FINANCER) (or FINANCER plus
ADMIN for override access), reusing the existing require_role function in
backend/app/platform/authorization/services.py. Do not write a second
authorization mechanism.

Frontend: a new "Financer Decision" panel on Advance Detail (Chunk 2) showing
every underwriting condition sourced from the rules configured in the Chunk 8
settings page — fully transparent, not summarized or pre-filtered, so the
financer sees the complete picture. Include the verification rollup already
linked from Chunk 2, Approve / Decline / Request More Info actions, and a
rationale selected from the Chunk 7 dropdown pattern, not a free-text field.

Role gating: only FINANCER can act on this panel and record a decision.
Compliance and Operations/RM see it read-only. Admin full. Developer read-
only masked.
```

Chunk 7 — Maker-checker approvals

Centrex reference: p.10 §9 "Notes / Underwriter Commentary" (structured, categorized notes — the source for the reason-code dropdown pattern here), and p.16 §15's "record which rule/data caused a pass, warning or rejection."

```
Backend: a new pending-change record — proposer, proposed value, status, and (on resolution) approver, which must be a different user than the proposer, enforced server-side. Applies to two existing surfaces that currently write directly and need to write a PENDING record instead: compliance decisions (find the service currently backing screening_review.py's write path) and qualification-rule changes (qualification.py / qualification_enums.py, currently written directly from QualificationCriteriaPage.tsx's backend).

Block self-approval at the service layer, not just by hiding a button in the UI — the same client-side-only mistake already found and fixed once in this codebase (a screening-review endpoint) must not be repeated here.

Also build: decision rationale fields, both here and on the Chunk 6 Financer panel, must be a dropdown selected from an admin-managed, extensible list of reason codes — not free text. Search for an existing reason-code entity before adding one; none was found in this session's exploration, so this is likely new.

Frontend: a new Approvals queue page listing every pending change, plus a "Pending Approval" badge shown inline on whichever screen the change originated from (QualificationCriteriaPage.tsx, and the compliance decision UI on the Company Dashboard).

Role gating: Compliance and Admin can propose. Admin can approve a change proposed by someone else (never their own). Financer and Operations/RM read-only. Developer read-only masked.
```

Chunk 8 — Verification & re-verification settings

Centrex reference: p.16 §15 "Underwriting Conditions, Decision Logic and Financier Submission" (condition status values: Pending/Passed/Failed/Waived/Needs Review).

New admin-only settings page, same pattern as the existing QualificationCriteriaPage.tsx — find and reuse that page's exact versioning mechanism on the backend rather than building a new one.

Shows an editable table mapping check type to vendor, seeded from this source list: PAN/CIN/GSTIN/Udyam → Decentro/Finpass; e-way bill → Surepass; commercial invoice → Finpass Invoice Analyser (mark this one provisional in the UI — its actual capability is unconfirmed); IEC/e-RCMC/export licences/restricted authorisations/eCoO → DGFT, plus Surepass for IEC specifically; Indian KYB → Surepass/Sandbox/Trulioo; US KYB → Middesk/Trulioo.

Also an editable re-verification cadence setting per risk tier. Do NOT hardcode specific tiers or windows ("6 years," "6 months" were discussion examples, not decisions) — build the settings UI generic enough to hold whatever tiers and windows get configured later.

Role gating: ADMIN gets full read/write. Every other role gets no route to this page at all — not a disabled or read-only view, no access.